ProTracer: Proprioception-Guided Failure Diagnosis in Robot Manipulation

Chang Dong[1], Mehdi Hosseinzadeh[1], King Hang Wong[1], Lingqiao Liu[1], Francois Fraysse[2], Feras Dayoub[1], and Minh Hoai[1,†]

https://protracer-failure.github.io

***Abstract*— This paper presents a comprehensive framework for robot manipulation failure analysis that includes binary failure detection, failure categorization, explanation generation, and the additional capability of *failure onset localization*, which aims to identify the earliest moment at which a robot execution deviates from a valid task-completion trajectory and is ultimately followed by task failure. To address these tasks, we propose ProTracer, a training-free framework that leverages existing Vision-Language Models (VLMs) together with proprioceptive signals for failure analysis. Our method uses proprioceptive dynamics to identify temporally informative action boundaries and converts richer robot-state signals into structured natural-language descriptions that can be jointly analyzed together with visual observations by the VLM. This design combines the temporal precision of proprioceptive signals with the multimodal reasoning capabilities of modern VLMs without requiring additional model training. We further introduce *FailTime*, a benchmark with synchronized visual and proprioceptive observations for evaluating conventional failure diagnosis tasks as well as failure onset localization. Experiments demonstrate that ProTracer achieves strong performance across both conventional failure diagnosis tasks and the newly introduced failure onset localization task, highlighting the importance of proprioceptive reasoning for fine-grained temporal failure analysis.**

I. Introduction

Failure diagnosis is important for robot learning and deployment, supporting applications such as data filtering, policy improvement, recovery planning, and safety analysis. However, existing approaches focus primarily on *whether* and *what* went wrong—detecting rollout failures, categorizing failure types, or explaining failure outcomes [1], [2], [3], [4], [5]—while largely overlooking *when* the failure first occurred. This distinction is important because manipulation failures may occur well before the end of a complete rollout, with the robot continuing to execute for many seconds after an initial mishap. Localizing failure onset enables more precise temporal credit assignment, recovery of successful trajectory prefixes from failed demonstrations, and finer-grained analysis of robot behavior.

We therefore study failure diagnosis as a joint problem that retains the existing diagnostic tasks and extends them to capture *when* a failure begins. Specifically, we consider failure detection, failure categorization, and explanation generation, together with *failure onset localization*. We define failure onset localization as the task of retrospectively identifying, given a completed manipulation rollout, the earliest moment at which the execution begins to lead to eventual task failure. As illustrated in Fig. 1, the failure onset may be the moment the gripper releases prematurely, rather than a later moment when the consequence becomes visually apparent or the task is ultimately declared unsuccessful.

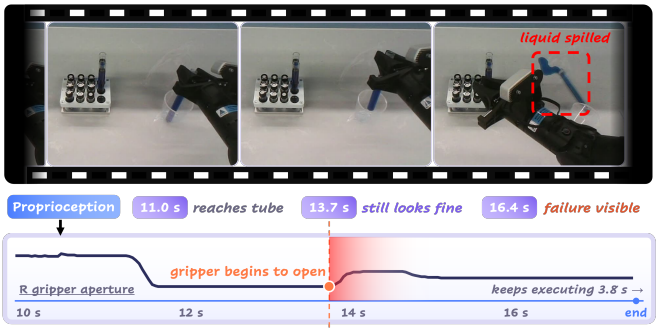


Fig. 1. **Failure onset localization.** The gripper releases prematurely at 13.7 s while the scene still looks normal; the spill only becomes visible at 16.4 s, and the robot keeps executing afterwards. ProTracer localizes this onset by using proprioception to pinpoint when the failure begins, and vision to identify what went wrong.

Importantly, this task cannot be adequately addressed by existing methods for online error or anomaly detection [6], [7], [8], [9], [10]. First, online methods operate only on observations available up to the current time, where early failure signals can be inherently ambiguous: apparent errors may later be recovered, while subtle errors may become recognizable only through their subsequent consequences. Consequently, the first alarm of an online detector is an unreliable onset estimate: a low threshold fires on transient deviations, a high one only after the failure is already visible, and neither marks where the execution actually went wrong. Second, online methods often rely on task-specific models of nominal execution, such as expected trajectories, task graphs, or learned distributions of successful behavior, requiring substantial task-specific data and training. In contrast, retrospective analysis can leverage the complete rollout and its known outcome to disambiguate earlier events without explicitly modeling every valid execution. Furthermore, access to the complete rollout enables joint reasoning over observations before and after a candidate

†Corresponding author.
[1]Australian Institute for Machine Learning and [2]School of Allied Health and Human Performance, Adelaide University, Australia.
Emails: Chang Dong, Mehdi Hosseinzadeh, King Hang Wong, Lingqiao Liu, Francois Fraysse, Feras Dayoub, Minh Hoai.

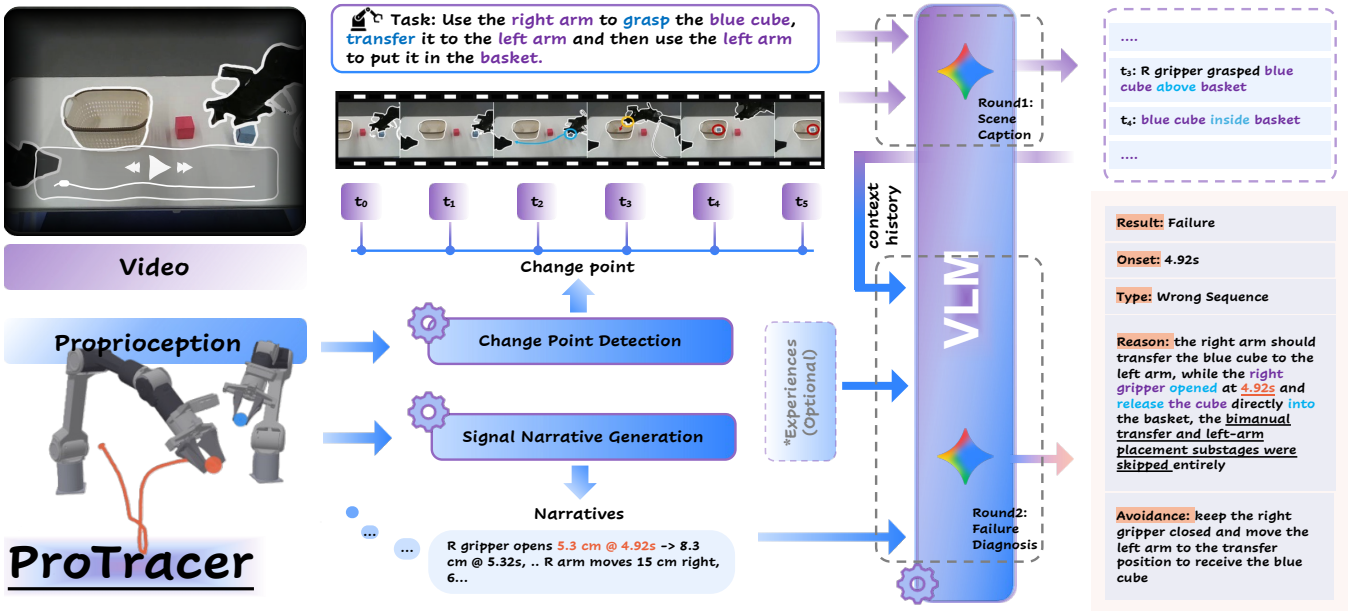


Fig. 2. **ProTracer Framework** for comprehensive robotic manipulation failure diagnosis, including failure detection, categorization, explanation, avoidance, and onset localization. It leverages proprioception for both keyframe selection and structured signal narratives generation. A VLM then jointly analyzes these multimodal inputs to infer the execution context and output the final diagnosis.

failure onset, making it possible to leverage the multimodal reasoning capabilities of pretrained VLMs for training-free failure analysis.

Despite this potential, leveraging VLMs for failure onset localization is non-trivial, as the critical transitions associated with failure onset are often visually subtle and temporally brief. Dense video prompting can overwhelm VLMs with redundant observations [2], [3], [5], while sparse visual sampling risks missing short, contact-rich transitions [4]. Our key observation is that manipulation failures are not purely visual phenomena, but are also reflected in the robot's execution dynamics. Gripper states and end-effector motion provide temporally precise cues to action transitions that can be difficult to infer from video alone, and such robot-state signals are routinely available in manipulation benchmarks and large-scale robot datasets [11], [12], [13], [14], [15]. Proprioceptive signals can therefore complement the visual reasoning capabilities of VLMs, providing fine-grained temporal cues for failure onset localization while also benefiting conventional failure diagnosis.

Technically, realizing this complementarity requires integrating proprioceptive signals into VLM-based reasoning. VLMs offer strong visual-language reasoning and semantic knowledge, but are not designed to interpret dense arrays of raw proprioceptive values. Training a new multimodal foundation model over vision, language, and robot-state trajectories is also impractical given the computational cost and limited annotated failure data. To address these challenges, we propose ProTracer, a training-free framework that integrates proprioception into existing VLMs in two complementary ways. First, proprioceptive dynamics identify temporal boundaries corresponding to meaningful action transitions, focusing visual analysis on informative rollout moments. Second, richer robot-state signals are converted into structured natural-language descriptions that can be jointly analyzed with visual observations. Together, these mechanisms combine the temporal precision of proprioception with the semantic reasoning capabilities of existing VLMs. When a small calibration set is available, ProTracer can further leverage prediction errors as reflective experience in future prompts, providing lightweight adaptation through in-context learning and verbal reflection [16], [17], [18].

To enable systematic evaluation, we also introduce *FailTime*, a robot manipulation benchmark with failure-onset annotations and synchronized visual and proprioceptive observations, combining re-annotated manipulation episodes with newly collected long-horizon real-world executions. Experiments show that modern VLMs already perform relatively well on binary failure detection under standard video prompting, while precise failure onset localization remains substantially harder. By integrating proprioception into VLM reasoning, ProTracer enables sub-second failure onset localization in the zero-shot setting while also improving conventional diagnosis tasks. Ablations further show that proprioception is critical for temporal localization, while visual evidence remains essential for semantic understanding.

In summary, our contributions are three-fold: **1)** We introduce *failure onset localization*, a new task for identifying the earliest failure-relevant deviation. **2)** We propose ProTracer, a training-free framework that leverages proprioception for action-boundary localization and structured multimodal VLM prompting. **3)** We release *FailTime*, a manipulation benchmark with failure-onset annotations and synchronized visual and proprioceptive observations.

II. Related Works

Our work relates to prior research on robot failure analysis, online failure detection, temporal localization, proprioceptive

reasoning, and VLM-based reasoning and adaptation.

Robot Failure Analysis has been studied through failure explanation, communication, correction, and recovery [19], [20]. Recent works leverage foundation models and VLMs for failure diagnosis using multisensory summaries, video-QA supervision, visual symbols, or keyframe prompting [1], [2], [3], [5], [4], and for replanning [21], recovery [22], and execution monitoring [23].These approaches primarily characterize whether and what went wrong, but not when the failure began.

Online Failure Detection. Online error or anomaly detectors [6], [7], [8], [9] operate causally on partial observations, where early failure evidence can be ambiguous and reliable detection may occur substantially after the actual onset. They also often require task- or policy-specific training data, nominal execution models, or threshold calibration. In contrast, we formulate onset localization as retrospective diagnosis, leveraging the complete rollout and outcome for more reliable temporal reasoning, which we empirically validate against online detectors in Sec. V-D.

Temporal Localization. Temporal video grounding and event spotting [24], [25], [26] localize visible events given text queries or predefined categories, whereas failure onset is the earliest deviation leading to eventual failure, conditioned on the task. Unlike VLM-based robot analysis that relies on dense or visually selected frames [2], [3], [4], [5], we use proprioceptive dynamics to locate action transitions.

Proprioception-Guided Reasoning. Proprioceptive measurements such as gripper state, end-effector motion, and actuator dynamics are widely available in robot manipulation systems and datasets [11], [12], [13], [14], [15], and have been used for control, manipulation learning, reward estimation, and trajectory generation [27], [28], [29], [30]. We instead study their role in post-hoc VLM-based failure reasoning, using gripper and end-effector dynamics to identify action boundaries and converting richer robot-state signals into VLM-compatible descriptions.

VLMs and Prompt-Level Adaptation. Our training-free framework builds on recent advances in VLM reasoning and prompt-level adaptation. VLMs have enabled language-conditioned planning, generalist policies, and visual reasoning in robotics [31], [32], [33], [34], [35]. In-context learning enables pretrained models to adapt from examples without parameter updates [16], while verbal reflection allows models to accumulate textual experience from previous errors [18], [36]. Building on these ideas, we convert prediction errors on a small calibration set into textual lessons that guide subsequent failure diagnosis without VLM fine-tuning.

III. Problem Formulation & Proposed Approach

A. Diagnostic Setting

We consider post-hoc failure diagnosis for robot manipulation episodes. Each episode contains a task instruction (e.g., "Use the right arm to grasp the blue cube and place it into the basket"), one or more RGB video streams, and synchronized proprioceptive signals such as gripper states and end-effector poses. The objective is to determine whether the episode succeeds or fails and, for failed episodes, predict the failure type, generate an explanation and avoidance suggestion, and localize the failure onset moment.

Failure onset localization is the new diagnostic capability introduced in this work. Intuitively, the onset is the earliest moment at which the robot execution deviates from a valid task-completion trajectory in an episode that ultimately fails. We interpret an execution as a sequence of semantically meaningful action stages inferred from visual and proprioceptive evidence, such as reaching, grasping, lifting, transporting, and placing. Since the analysis is post-hoc and the full episode is available, temporary mistakes that are later corrected remain part of a valid execution; for example, a failed grasp followed by a successful retry is not treated as a failure onset. Thus, failure onset is defined only for episodes that ultimately fail, and corresponds to the earliest deviation after which the execution no longer returns to a successful task-completion trajectory.

This deviation can occur in three ways. First, the robot may transition away from an unsuccessful action without achieving its intended outcome, such as moving to placement without successfully grasping the object. Second, it may execute an action inconsistent with the task objective, such as placing the object in the wrong location or moving to an unrelated workspace region. Third, it may remain in the same action stage beyond an acceptable timeout without meaningful progress, such as repeatedly attempting to grasp without stable contact; in this case, the onset is the moment the timeout threshold is exceeded.

These rules provide an objective protocol for annotating failure onset without requiring exhaustive enumeration of all valid execution strategies or a complete task graph. Instead, the annotator only needs to judge whether the current execution remains consistent with *some* valid task-completion trajectory. This is practical for post-hoc analysis: humans can typically identify when a robot is no longer "on track" without enumerating every possible valid execution.

B. ProTracer Overview

Fig. 2 depicts ProTracer, a training-free VLM framework for post-hoc failure diagnosis that uses proprioception in two complementary ways: gripper and end-effector dynamics drive change-point detection to select informative transition moments and visual evidence, and robot-state measurements are converted into structured natural-language descriptions that the VLM analyzes jointly with the selected frames.

1) Proprioceptive Analysis for Key Moment Detection: We use proprioceptive signals to identify candidate key moments in the execution. Joint and gripper states are converted into an 8D end-effector trajectory consisting of 3D position, quaternion orientation, and gripper aperture. For change-point detection, we focus on gripper dynamics and end-effector motion, which provide the most informative cues for manipulation transitions; the full 8D signal is used later for natural-language narrative generation.

Suppose the robot has K arms. Let v_t^k denote the gripper-aperture velocity of arm k at time t, and let m_t^k denote

its denoised end-effector motion signal computed from consecutive positions. For each arm, we encode two discrete indicators: $\text{sign}(v_t^k) \in \{-1, 0, 1\}$ and $\mathbb{1}[m_t^k > 0] \in \{0, 1\}$. We discard magnitudes because primitive boundaries are primarily characterized by discrete changes in grasp state and motion activity rather than movement amplitude. For an episode of length T, this yields a symbolic matrix $X \in \mathbf{\Sigma}^{T \times 2K}$, where x_t^c denotes channel c at time t and $\mathbf{\Sigma}$ is the induced discrete alphabet.

For a candidate segment $[s, e)$, we define the modal-Hamming cost as

$$C(s, e) = \sum_{c=1}^{2K} \min_{v \in \Sigma_c} \sum_{t=s}^{e-1} \delta(x_t^c \neq v). \tag{1}$$

where $\delta(\cdot)$ is the indicator function and $\mathbf{\Sigma}_c$ is the alphabet of channel c. This cost measures how well a segment can be represented by a single stable symbolic state. Stable manipulation primitives yield low cost, while transitions between primitives produce higher inconsistency.

We apply PELT [37] to detect candidate change points, then greedily prune weak boundaries to satisfy a fixed temporal budget. For a boundary b_i with neighboring boundaries b_{i-1} and b_{i+1}, the removal cost increase is $S(b_i) = C(b_{i-1}, b_{i+1}) - C(b_{i-1}, b_i) - C(b_i, b_{i+1})$. A small increase indicates that adjacent segments have similar proprioceptive states and can be merged with little loss. We iteratively remove the boundary with the smallest increase until at most B temporal phases remain (B=8 on FailTime-Short and 39 on FailTime-Long, with 6.8 and 19.5 phases retained on average respectively). The retained boundaries define the selected keyframes and temporal intervals for downstream evidence construction.

2) Proprioception-Guided VLM Inference: Given the detected boundaries, ProTracer extracts representative visual keyframes and converts proprioceptive signals into structured natural-language descriptions for joint reasoning with visual observations. Each interval is represented by keyframes, timestamps, and a rule-based sensor narrative summarizing the robot dynamics. These narratives describe gripper actions (opening, closing, holding), end-effector motion, and wrist rotation using displacement, motion direction, and rotation consistency, while merging short transient runs to reduce noise. They provide localized descriptions of robot behavior between sparse visual observations. We deliberately use deterministic rules rather than a learned captioner: the narration is hallucination-free, and its vocabulary consists of embodiment-agnostic physical quantities (aperture, displacement, rotation), which is why the same rules transfer unmodified to a different platform in Sec. V.

VLM inference proceeds in two stages. First, the VLM analyzes selected keyframes from the available camera views, such as head-view and wrist-view images, and generates concise scene-state descriptions. To improve grounding consistency, we provide a task-specific reference vocabulary derived from the task instruction, covering relevant objects, actions, gripper states, gripper–object relations, and object–object relations. Second, the VLM receives the task instruction, visual keyframe descriptions, proprioceptive descriptions, failure taxonomy, and optional reflective experiences. The visual descriptions capture sparse scene states, while the proprioceptive descriptions characterize the physical transitions between them. From this combined prompt, the VLM constructs a concise event-level interpretation and predicts the final diagnosis.

3) Optional Reflective Learning for Few-shot Adaptation: The preceding pipeline is training-free and can be applied directly to unseen robot embodiments and manipulation tasks.

When a small annotated calibration set is available for the target environment or task, ProTracer can adapt to recurring error patterns that general-purpose VLM reasoning may miss without updating any weights. To use this information without fine-tuning the VLM, we introduce an optional reflective memory for lightweight in-context adaptation.

Given a calibration set, the VLM first predicts the failure verdict, onset time, and failure type for each episode. The prediction is then paired with the ground-truth labels and returned to the same VLM through a reflection prompt. The VLM analyzes the discrepancy and generates a textual lesson, such as a common onset-localization error, a confusion between similar failure categories, or a mismatch between visual and proprioceptive evidence.

These lessons are accumulated into a reflective experiences and prepended to future inference prompts. To keep the memory compact, the VLM may *ADD* new lessons, *MERGE* redundant observations, *UPDATE* conflicting rules, or *COMPRESS* the memory under a fixed word budget. Fig. 3 illustrates two real-world lessons successfully accepted into memory via in-context reflection. A candidate lesson is accepted only if re-running inference on the corresponding calibration episode with the updated memory corrects the original prediction; otherwise, it is discarded. This acceptance procedure is used only during adaptation. At test time, the memory is fixed and no ground-truth labels are used.

No VLM weights are updated, and no auxiliary reflector model is trained; adaptation occurs entirely through prompt-level contextual guidance. When no calibration data is available, the reflective memory is empty and ProTracer reduces to the training-free proprioception-guided inference pipeline.

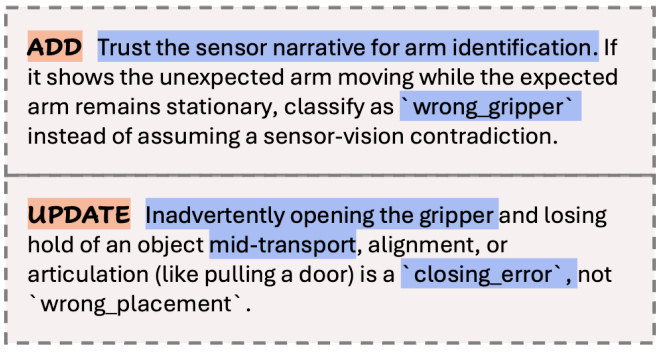


Fig. 3. **Examples of learned experiences.**

TABLE I

RESULTS ON FAILTIME-SHORT (AVG 12 S , MAX 37 S) AND FAILTIME-LONG (AVG 54 S , MAX 120 S). VARIANTS MARKED * ADDITIONALLY RECEIVE RAW PROPRIOCEPTIVE SIGNALS; THE ONE MARKED † IS TRAINED ON THE VIFAILBACK TRAINING SPLIT, HENCE NOT ZERO-SHOT.

Model	FailTime-Short							FailTime-Long						
	Failure Onset			Fail. Detect.	Fail. Type	Fail. Reason	Fail. Avoid.	Failure Onset			Fail. Detect.	Fail. Type	Fail. Reason	Fail. Avoid.
	MAE↓	@0.5s	@1s					MAE↓	@0.5s	@1s				
Failure Models														
ViFailback-8B† [5]	1.10	46.06	67.49	97.80	57.64	62.56	40.96	14.20	5.26	7.02	68.00	12.28	22.72	17.02
KITE [4]	1.75	14.53	33.25	96.26	35.71	50.86	36.13	12.26	1.75	8.77	75.00	31.58	33.42	29.47
Robotic Models														
Robotics-ER 1.6 [38]	1.63	14.29	36.21	95.38	34.48	49.06	24.06	6.50	14.91	34.21	**88.00**	37.72	38.54	34.78
Robotics-ER 1.6* [38]	1.42	33.50	44.83	96.70	35.71	48.89	26.88	5.82	17.54	30.70	83.00	40.35	38.84	26.45
General Models														
Gemini-3.1-pro	1.32	27.83	55.42	95.82	49.51	54.30	41.26	10.22	8.77	21.05	67.00	38.60	35.53	31.23
Gemini-3.1-pro*	0.90	48.03	73.15	98.24	60.34	61.92	44.90	10.11	24.56	31.58	68.00	38.60	35.86	29.65
Ours														
ProTracer	0.59	71.18	81.03	**99.34**	72.91	**69.40**	50.87	4.06	40.35	49.12	87.00	57.89	49.19	40.09
ProTracer+Exp.	**0.47**	**77.34**	**83.99**	98.68	**77.83**	69.20	**51.24**	**3.27**	**45.61**	**50.88**	**88.00**	**59.65**	**50.16**	**40.18**

IV. FAILTIME DATASET

To evaluate failure onset localization, we introduce *FailTime*, a robot manipulation dataset with frame-level onset annotations and detailed failure diagnoses, including 123 distinct manipulation tasks. It has two components. *FailTime-Short* builds on the full ViFailback benchmark [5] and a subset of its training data, yielding 1,539 annotated episodes, including 579 successful executions. *FailTime-Long* is a newly collected set of 100 long-horizon real-world episodes (57 failures and 43 successes) on two bimanual platforms, a general-purpose ALOHA system and a cost-effective SO-101 setup. Its episodes average 54 s (max 120 s), much longer than the 12 s average (max 37 s) of FailTime-Short, so the onset must be located among far more candidate boundaries. It is designed to test cross-dataset transfer under different embodiments, scenes, and longer task horizons.

All episodes are annotated using the failure onset formulation in Sec. III-A. We develop an annotation interface for jointly inspecting synchronized video streams and aligned proprioceptive signals. Annotators are also given high-level task decompositions and example successful executions, while independently judging when the execution meaningfully deviates from a plausible successful trajectory.

Beyond onset timestamps, we refine the original failure annotations by expanding the taxonomy from four to ten categories and correcting inconsistent reason and avoidance labels. The ten categories (wrong sequence, wrong object, wrong gripper, wrong placement, closing error, open error, translation error, orientation error, avoidable intervention, and unavoidable intervention), each failed episode receives one primary category, and each category has an explicit onset rule defining the earliest action that commits the robot to that failure. Six annotators with robotics and machine learning backgrounds spent over 120 hours annotating onset timestamps and failure categories. The resulting labels show strong consistency: average pairwise agreement is 0.94 for failure categories, mean onset disagreement is 50ms, and 99.6% of onset disagreements fall within 0.5s.

V. EXPERIMENTS

A. Experimental Setup

We perform a variety of experiments using ProTracer with Gemini-3.1-pro as the underlying VLM backbone throughout. Performance is evaluated on the FailTime-Short and FailTime-Long benchmarks. For the optional reflective-adaptation variant, we construct the memory from 50 episodes randomly sampled from the unused portion of the training split, with no overlap with either the FailTime-Short or FailTime-Long evaluation set. We perform three reflective epochs. Sensitivity to both choices is analyzed in Table IV. Since ProTracer runs offline, it is not latency-constrained; for reference, an episode costs about $0.09 and takes 40–60 s on Gemini-3.1-pro, dominated by the two VLM calls.

Baselines. We compare against several classes of baselines, most of which share the same Gemini-3.1-pro backbone as our method. First, we compare against Gemini-3.1-pro itself as a general-purpose VLM baseline, which uses standard dense prompting of multi-view video frames sampled at 1 fps. Second, we compare against Robotics-ER 1.6, a robotics-oriented model tuned by robotic data. Third, we compare against two methods specifically designed for robot failure diagnosis: KITE and ViFailback-8B. KITE is also based on Gemini-3.1-pro and uses optical-flow-based keyframe selection. For KITE, we preserve the original configuration whenever possible and modify only the prompting format to produce onset timestamps and the required diagnosis outputs. ViFailback-8B [5] is a trained failure-analysis model originally developed for the ViFailback benchmark. Since it is trained on the original ViFailback training split, it is not a zero-shot baseline, unlike our approach and the other methods in the comparison. We additionally evaluate variants of these methods that receive proprioceptive signals as CSV files containing textual descriptions of feature types together with the corresponding numerical values. These variants are marked with an asterisk ($*$); ViFailback-8B, which is trained on the ViFailback training split, is marked with a dagger (†).

Metrics. For failure detection and failure-type classification, we report accuracy. Since some ambiguous episodes can contain multiple plausible failure types, a type prediction is

considered correct if it matches any label in the ground-truth set. For failure onset grounding, we report tolerance-based accuracy (%) [26], [39], denoted $\text{acc}@\delta$, where a prediction is correct if the absolute onset error is less than δ. We use $\delta \in \{0.5s, 1s\}$ and also report the mean absolute error (MAE) in seconds, computed only over correctly detected failure episodes. For failure reason and avoidance prediction, we use GPT-5 as an LLM judge following the rubric of ViFailback [5]. For runtime detectors, we additionally report the area under the ROC curve (AUROC), together with the true-positive rate (TPR) and true-negative rate (TNR) at the calibrated operating threshold.

B. Main Results

Results on FailTime-Short. Table I shows that the primary challenge in *FailTime-Short* is not binary failure detection, but temporally precise onset grounding. Dense-frame and failure-analysis baselines already achieve high failure detection accuracy, with all models exceeding 95%, whereas their onset accuracy varies widely. Access to proprioception already helps: simply appending raw proprioceptive signals to the same dense-frame Gemini-3.1-pro baseline (marked with * in Table I) reduces MAE from 1.32 s to 0.90 s and raises $acc@0.5s$ from 27.83% to 48.03%, even without any change to the prompting scheme. ProTracer improves further, reaching 71.18% $acc@0.5s$ and 0.59 s MAE zero-shot, and 77.34% and 0.47 s with reflective experiences. Proprioception is thus the source of the temporal signal, but how it is used matters as much: selecting key frames at proprioceptive change points and converting the signals into structured narratives further raises acc@0.5 s from 48.03% to 71.18% and lowers MAE from 0.90 s to 0.59 s over raw sensor values alone, with Sec. V-C isolating each component. Compared with ViFailback-8B, which is trained on the original ViFailback data, ProTracer still achieves substantially better strict temporal localization without any training. KITE, which likewise feeds a VLM with selected key frames but chooses them from visual cues, trails far behind (14.53% $acc@0.5s$); since replacing Sign-CPD with an optical-flow selector inside ProTracer still yields 66.50% (Table III), the gap stems mainly from the proprioceptive narratives rather than frame selection alone. ProTracer also improves failure-type classification to 77.83%, and gives the best failure reason and avoidance scores in Table I as well.

Results on FailTime-Long. We further verify ProTracer on long-horizon executions using *FailTime-Long* in different embodiments and scenes (Table I). All methods degrade in onset localization on the longer horizons, but ProTracer degrades the least, remaining the best on every onset and diagnosis metric while matching the best baseline on failure detection. ViFailback-8B, trained on ViFailback data and thus in-distribution for *FailTime-Short*, drops from 46.06% to 5.26% at $acc@0.5s$, whereas training-free ProTracer retains 40.35% (45.61% with experiences). Fig. 4 shows a qualitative comparison between ViFailback-8B and ProTracer. Interestingly, Robotics-ER 1.6, whose training includes robot sensor data [38], degrades far less than Gemini-3.1-pro and is the strongest baseline on most metrics, again underscoring the value of proprioception, which ProTracer supplies to an off-the-shelf VLM without training.

C. Ablations and Analysis

Modality Ablation. Table II shows that proprioception and vision play complementary roles. Removing sensor input causes the largest drop in strict onset localization: $\text{acc}@0.5s$ decreases from 71.18% to 30.79%, and MAE doubles from 0.59 s to 1.18 s. This confirms that gripper and end-effector change points provide the temporal structure needed for sub-second grounding. Meanwhile, failure detection remains high (98.24%) and type accuracy drops only moderately (57.39%), so vision alone still supports semantic judgement. Removing vision instead collapses failure-type accuracy to 15.02%: without visual context, the model cannot determine what object-level failure occurred. These results support the design principle of ProTracer: sensors determine *when* to inspect, while vision-language reasoning determines *what* failed.

Key-frame Selection. ProTracer selects key frames with Sign-CPD, a change-point detector over proprioceptive signals. Two questions follow: whether proprioception is truly necessary for this purpose or video-based cues such as optical flow could provide similar temporal information, and whether the symbolic sign-based formulation matters or a continuous-valued CPD on the same raw channels would do. Tab. III answers both by swapping the key-frame selector inside the same pipeline while keeping everything else fixed. Sign-CPD is best on every diagnosis metric: onset accuracy within 0.5s improves from 66.50% (optical flow) and 66.26% (continuous CPD) to 71.18%, MAE from 0.73s and 0.79s to 0.59s, and failure-type accuracy from 64.78% and 66.50% to 72.91%. To explain the gain, we

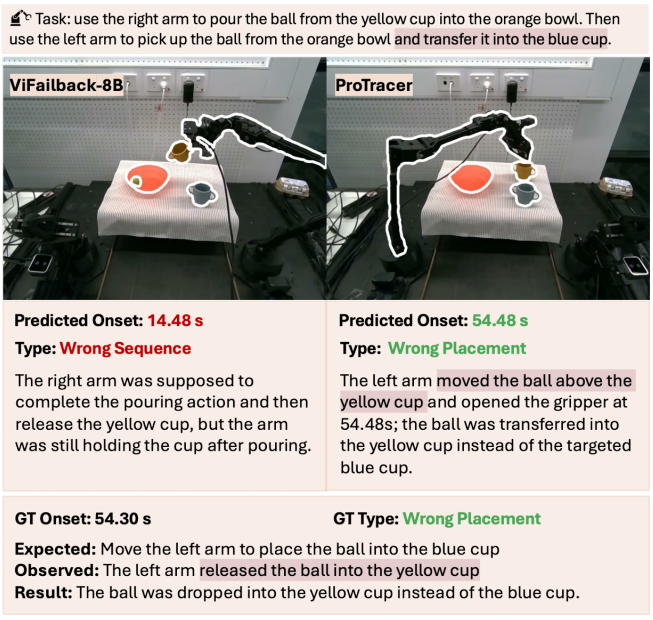


Fig. 4. **Qualitative comparison on the FailTime-Long.** While ViFailback-8B incorrectly grounds the failure onset to an earlier stage, ProTracer precisely localizes the true onset.

TABLE II

MODALITY ABLATIONS ON FAILTIME-SHORT.

Model	Failure Onset			Fail. Detect.	Fail. Type
	MAE↓	@0.5s	@1s		
ProTracer (Sign-CPD)	**0.59**	**71.18**	**81.03**	**99.34**	**72.91**
w/o sensor	1.18	30.79	59.85	98.24	57.39
w/o vision	1.88	23.65	48.52	90.33	15.02

TABLE III

KEY-FRAME SELECTORS ON FAILTIME-SHORT.

Selector	Failure Onset			Fail. Detect.	Fail. Type
	MAE↓	@0.5s	@1s		
ProTracer (Optical Flow)	0.73	66.50	76.11	98.68	64.78
ProTracer (Continuous CPD)	0.79	66.26	77.34	98.90	66.50
ProTracer (**Sign-CPD**)	**0.59**	**71.18**	**81.03**	**99.34**	**72.91**

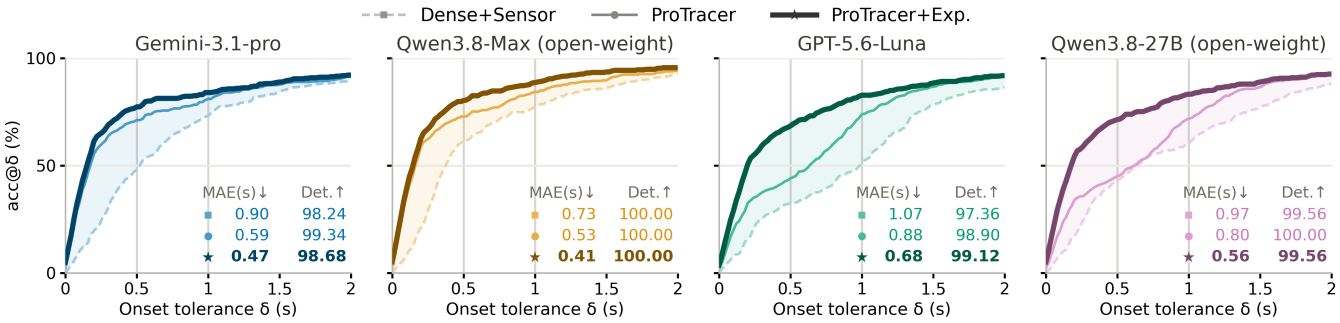


Fig. 5. **Generalization across VLM backbones on FailTime-Short.** Dense-frame prompting with raw proprioception vs. ProTracer and ProTracer+Exp; the reflective memory is learned with Gemini-3.1-pro and reused unchanged on the other backbones.

TABLE IV

CALIBRATION-SET SIZE AND REFLECTIVE EPOCHS.

Setting	MAE↓	@0.5s↑	@1s↑	Det. ↑	Type ↑
ProTracer *(w/o Exp.)*	0.59	71.18	81.03	**99.34**	72.91
Calibration-set size					
25 eps	0.55	73.65	81.28	98.68	76.85
50 eps	**0.47**	**77.34**	**83.99**	98.68	**77.83**
100 eps	**0.47**	75.62	82.51	98.68	76.60
Reflective epochs (50 eps)					
1 epoch	0.56	74.38	81.77	98.68	71.18
2 epochs	0.52	74.14	83.74	98.90	77.09
3 epochs	**0.47**	**77.34**	**83.99**	98.68	**77.83**

TABLE V

RUNTIME FAILURE DETECTORS ON FAILTIME-SHORT.

Detector	Failure Detection			Failure Onset		
	AUROC	TPR	TNR	MAE↓	@0.5s	@1s
RND-OE [9]	0.57	44.83	79.59	5.03	0.74	2.96
FIDeL [10]	0.60	34.73	95.92	5.27	0.25	1.97
RynnValue-8B [40]	0.85	38.18	93.88	2.61	1.72	5.67
GRU (Onset Supervised)	0.74	71.43	65.31	2.08	18.97	33.99
ProTracer (zero-shot)	–	**99.26**	**100.00**	**0.59**	**71.18**	**81.03**

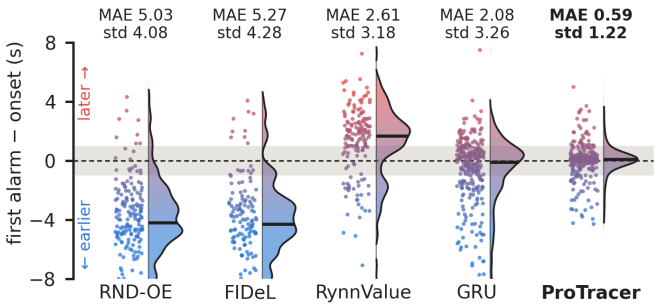


Fig. 6. **First-alarm error relative to the annotated onset on FailTime-Short** (alarmed episodes only, clipped to ±8 s; band: 1 s tolerance).

ran a separate boundary-alignment experiment: we manually annotated action boundaries on 50 *FailTime-Short* episodes with rich action transitions and measured how many of them each selector recovers within 0.5s. Sign-CPD recalls 69.68%, versus 40.80% for optical flow and 55.28% for continuous CPD, indicating that its key frames fall closer to the true primitive boundaries and suggesting that such boundaries are better captured by discrete state switches than by visual motion or amplitude changes.

Generalization across VLM backbones. To verify that the onset-localization gains are not tied to **Gemini-3.1-pro**, Fig. 5 repeats the comparison between dense-frame prompting with raw proprioception (Dense+sensor), ProTracer, and ProTracer+Exp with three additional backbones: **Qwen3.8-Max**, **GPT-5.6-Luna**, and **Qwen3.8-27B**. ProTracer improves over Dense+sensor on the three additional backbones for onset localization with lower MAE, confirming that the benefit comes from structuring proprioception into boundary evidence and narratives rather than from access to raw sensor values. The consistent separation of the curves across backbones further shows that the improvement holds for models of different capacities.

Sensitivity to calibration-set size and reflective epochs. Table IV varies the calibration-set size and the number of reflective epochs. Every configuration improves onset performance over ProTracer without experiences, so the benefit of reflective memory does not depend on a particular setting. Using more than 50 calibration episodes brings no further gain, suggesting that a small calibration set suffices.

Reflective experience transfer. The reflective experience transfers in two ways. First, a memory built on *FailTime-Short* also improves *FailTime-Long* (Table I). Second, the same memory, learned with Gemini-3.1-pro and reused unchanged, consistently improves onset localization, with particularly large gains for the weaker models (Fig. 5). This suggests that reflective experience captures transferable knowledge rather than model-specific prompt tuning.

D. Comparison with Runtime Failure Detectors

Runtime failure detectors are designed to stop a policy, not to localize where an execution went wrong, but a natural question is whether their first alarm could serve as an onset estimate. Table V tests this on FailTime-Short by running each detector causally over every episode and taking its first alarm as the onset estimate; episodes without an alarm count as misses. We include the **RND-OE** uncertainty score [9], **FIDeL** [10] (statistical filter), a value-drop detector on a finetuned **RynnValue-8B** [40], and, as a strong reference, a **GRU** trained with onset labels on the FailTime-Short evaluation episodes themselves via 5-fold cross-validation, giving it in-distribution supervision that others do not use. All detectors are run policy-agnostically on frozen visual features and, following the original methods, thresholded to a 5% false-alarm rate on held-out successful demonstrations. Runtime alarms are poor onset estimates. The three label-free detectors, which are fit on successful rollouts only, reach at most 1.7% $acc@0.5s$ with MAEs of 2.6–5.3 s, and even the supervised GRU only 19.0% and 2.08 s, against 71.2% and 0.59 s for ProTracer without any training. Fig. 6 shows why the gap cannot be calibrated away: the two density-based detectors fire systematically early and the value model late, but with a spread of 3–4 s around their medians, so shifting every alarm of a detector by its own median offset, an oracle correction estimated on the test set, still leaves onset accuracy below 9% $acc@0.5s$ (18.7% for the GRU). Runtime detection and onset localization thus answer different questions. Runtime detectors decide when to stop under causal constraints, and doing so reliably requires nominal data for calibration and a tolerance for early alarms; ProTracer instead recovers where an execution went wrong from the full recording without any training, which makes it the better fit for post-hoc analysis and root-cause tracing.

VI. Conclusion

In this paper, we introduced failure onset localization as a new capability for robot manipulation failure diagnosis, together with FailTime, a benchmark with synchronized visual and proprioceptive observations for evaluating fine-grained temporal failure analysis. We further proposed ProTracer, a training-free framework that integrates visual and proprioceptive signals for multimodal failure reasoning using existing VLMs. Experimental results demonstrate strong performance across both conventional failure diagnosis tasks and the newly introduced onset localization task, highlighting the importance of proprioceptive reasoning for precise robot failure understanding.

ACKNOWLEDGMENT

The authors sincerely thank the additional annotators, Yichao Cai, Shuying Piao, and Lishan Yang, for their valuable contributions to the annotation and quality control of the FailTime benchmark. Their careful review and feedback helped improve the consistency and reliability of the annotations.

References

[1] Z. Liu, A. Bahety, and S. Song, "REFLECT: Summarizing robot experiences for failure explanation and correction," in *CoRL*, 2023.

[2] J. Duan *et al.*, "AHA: A vision-language-model for detecting and reasoning over failures in robotic manipulation," *arXiv preprint arXiv:2410.00371*, 2024.

[3] W. Lu, M. Ye, Z. Ye, R. Tao, S. Yang, and B. Zhao, "RoboFAC: A comprehensive framework for robotic failure analysis and correction," *arXiv:2505.12224*, 2025.

[4] M. Hosseinzadeh, K. H. Wong, and F. Dayoub, "KITE: Keyframe-indexed tokenized evidence for vlm-based robot failure analysis," in *ICRA*, 2026.

[5] X. Zeng *et al.*, "Diagnose, correct, and learn from manipulation failures via visual symbols," in *CVPR*, 2026.

[6] C. Agia *et al.*, "Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress," in *CoRL*, 2025.

[7] Q. Gu *et al.*, "SAFE: Multitask failure detection for vision-language-action models," in *NeurIPS*, 2025.

[8] R. Römer, A. Kobras, L. Worbis, and A. Schoellig, "Failure prediction at runtime for generative robot policies," in *NeurIPS*, 2025.

[9] C. Xu *et al.*, "Can we detect failures without failure data? uncertainty-aware runtime failure detection for imitation learning policies," in *RSS*, 2025.

[10] Q. Rolland, F. Mayran de Chamisso, and J.-B. Mouret, "Failure identification in imitation learning via statistical and semantic filtering," in *ICRA*, 2026.

[11] S. James, Z. Ma, D. R. Arrojo, and A. J. Davison, "RLBench: The robot learning benchmark & learning environment," *IEEE RA-L*, vol. 5, no. 2, pp. 3019–3026, 2020.

[12] T. Mu *et al.*, "Maniskill: Generalizable manipulation skill benchmark with large-scale demonstrations," *arXiv preprint arXiv:2107.14483*, 2021.

[13] W. Pumacay, I. Singh, J. Duan, R. Krishna, J. Thomason, and D. Fox, "The colosseum: A benchmark for evaluating generalization for robotic manipulation," *arXiv:2402.08191*, 2024.

[14] A. Khazatsky *et al.*, "DROID: A large-scale in-the-wild robot manipulation dataset," in *RSS*, 2024.

[15] A. O'Neill *et al.*, "Open X-Embodiment: Robotic learning datasets and rt-x models," in *ICRA*, 2024.

[16] T. Brown *et al.*, "Language models are few-shot learners," in *NeurIPS*, 2020.

[17] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," in *NeurIPS*, 2022.

[18] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *NeurIPS*, 2023.

[19] D. Das, S. Banerjee, and S. Chernova, "Explainable ai for robot failures: Generating explanations that improve user assistance in fault recovery," in *HRI*, 2021.

[20] C. DeChant, I. Akinola, and D. Bauer, "Learning to summarize and answer questions about a virtual robot's past actions," *Autonomous robots*, vol. 47, no. 8, pp. 1103–1118, 2023.

[21] M. Skreta, Z. Zhou, J. L. Yuan, K. Darvish, A. Aspuru-Guzik, and A. Garg, "Replan: Robotic replanning with perception and language models," *arXiv:2401.04157*, 2024.

[22] S. S. Raman *et al.*, "Cape: Corrective actions from precondition errors using large language models," in *ICRA*, 2024.

[23] E. Zhou *et al.*, "Code-as-monitor: Constraint-aware visual programming for reactive and proactive robotic failure detection," in *CVPR*, 2025.

[24] L. A. Hendricks, O. Wang, E. Shechtman, J. Sivic, T. Darrell, and B. Russell, "Localizing moments in video with natural language," in *ICCV*, 2017.

[25] K. Grauman *et al.*, "Ego4D: Around the world in 3,000 hours of egocentric video," in *CVPR*, 2022.

[26] A. Deliège *et al.*, "SoccerNet-v2: A dataset and benchmarks for holistic understanding of broadcast soccer videos," in *CVPR Workshops*, 2021.

[27] Y. J. Ma, S. Sodhani, D. Jayaraman, O. Bastani, V. Kumar, and A. Zhang, "Vip: Towards universal visual reward and representation via value-implicit pre-training," *arXiv:2210.00030*, 2022.

[28] A. Mandlekar *et al.*, "Mimicgen: A data generation system for scalable robot learning using human demonstrations," in *CoRL*, 2023.

[29] R. Hoque, A. Mandlekar, C. Garrett, K. Goldberg, and D. Fox, "Intervengen: Interventional data generation for robust and data-efficient robot imitation learning," in *IROS*, 2024.
[30] C. R. Garrett, A. Mandlekar, B. Wen, and D. Fox, "Skillgen: Automated demonstration generation for efficient skill learning and deployment," in *CoRL Workshop*, 2024.
[31] B. Ichter *et al.*, "Do as i can, not as i say: Grounding language in robotic affordances," in *CoRL*, 2023.
[32] B. Zitkovich *et al.*, "Rt-2: Vision-language-action models transfer web knowledge to robotic control," in *CoRL*, 2023, pp. 2165–2183.
[33] J. Liang *et al.*, "Code as policies: Language model programs for embodied control," in *ICRA*, 2023.
[34] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, "Voxposer: Composable 3d value maps for robotic manipulation with language models," in *CoRL*, 2023.
[35] B. Chen *et al.*, "Spatialvlm: Endowing vision-language models with spatial reasoning capabilities," in *CVPR*, 2024.
[36] G. Wang *et al.*, "Voyager: An open-ended embodied agent with large language models," *arXiv:2305.16291*, 2023.
[37] R. Killick, P. Fearnhead, and I. A. Eckley, "Optimal detection of changepoints with a linear computational cost," *J. Amer. Statist. Assoc.*, vol. 107, no. 500, pp. 1590–1598, 2012.
[38] Gemini Robotics Team, "Gemini robotics: Bringing ai into the physical world," *arXiv:2503.20020*, 2025.
[39] J. Hong, H. Zhang, M. Gharbi, M. Fisher, and K. Fatahalian, "Spotting temporally precise, fine-grained events in video," in *ECCV*, 2022.
[40] D. Huang *et al.*, "RynnValue: Scaling robotic value foundation models with temporal distance," *arXiv:2608.09853*, 2026.